Institute of
Space Technology

COMPUTER NETWORKS

Assignment 1 Report

C++ Socket Programming & a Multiuser Chat System

Hania Zafar	250901028
Farwa Zainab	250901069
Nayab Fatima	250901024
Aiman Atif	250901049
Warisha Bukhari	250901005

Course: Computer Networks
Instructor: Mr Faran Mahmood
Batch: AI-02 | Semester: Fall 2026
Submission Date: November 8, 2026

GitHub Repository: github.com/Farwa705/Group-Assignment-1-netlab

Table of Contents

1. Introduction 3
2. System Architecture - the net.hpp Helper 3
3. Program Implementations 4
4. Multi-User Chat System (Port 6000) 6
5. Bonus - Crashing the Web Server 7
6. Compilation Commands 9
7. Verification & Testing Evidence 9
8. Challenges We Faced 15
9. Requirements Checklist 16
10. Conclusion 17

1. Introduction

This report documents our work on Computer Networks Assignment 1, which focuses on socket programming in C++.

The purpose of the assignment was to help us understand how data actually moves over a TCP connection, why that stream of bytes does not automatically arrive as neat "messages," and how a server can be designed to either serve one client at a time, one thread per client, or many clients at once using select(). All six example programs plus the chat system were implemented, tested, and are documented below with screenshots as evidence. We also attempted the bonus task, which asked us to find and demonstrate a weakness in one of the provided servers.

Our full source code, the www/ folder for the web server, and every screenshot referenced in this report are available in our public GitHub repository, linked on the cover page and again in the references section at the end of this document.

2. System Architecture - the net.hpp Helper

Writing socket code that compiles on both Windows and Linux is difficult because the two platforms use different headers, different types for a socket handle, and different startup/cleanup calls. Rather than repeating #ifdef _WIN32 blocks inside every single program, we isolated all of that platform specific code into one shared header, net.hpp, and built every other file on top of a small, clean wrapper API.

On Windows, net::init() calls WSAStartup() and sockets are represented using the SOCKET type, closed with closesocket(). On Linux, the same functions compile down to plain integer file descriptors and the standard close() call. Every other file in the project only ever calls the wrapper functions below, so none of our actual programs including chat_server.cpp and chat_client.cpp need a single #ifdef of their own.

2.1 Wrapper functions

- net::init() / net::cleanup() — starts and shuts down the networking subsystem (a no-op on Linux, WSAStartup/WSACleanup on Windows)
- net::tcp_listen(port) — creates a socket, binds it to a port, and starts listening
- net::tcp_accept(listener) — blocks until a client connects, returns a socket for that client
- net::tcp_connect(host, port) — used by clients to connect to a server
- net::send_all(socket, data) — keeps calling send() until the entire string has gone out, since a single send() is not guaranteed to send everything at once
- net::recv_some(socket, buffer, length) — reads whatever bytes are currently available
- net::close_socket(socket) — closes a connection

This design meant that once net.hpp was working correctly, every program after it from the two line daytime server to the select()based chat server could be written as plain, portable C++ with no platform branching of its own.

3. Program Implementations

3.1 Daytime Server & Client (Port 9000)

What it does: This is the simplest possible TCP exchange. The server listens on port 9000, and the moment a client connects, it reads the current system time with ctime(), sends that single string back, and closes the connection immediatel.

How we tested it: We ran the server in one terminal and the client in a second terminal and confirmed that the client printed the exact date and time the server had sent, proving the connect/send/receive cycle worked end to end.

3.2 Echo Server & Client (Port 7000)

What it does: An iterative server: once a client connects, anything that client types is read into a buffer and sent straight back (“echoed”), and this repeats in a loop until the client disconnects or sends an empty line.

How we tested it: We typed several different lines into the client and verified that every single one came back from the server unchanged, confirming the send/receive loop was stable across multiple round trips, not just a single message.

3.3 Static Web Server (Port 8080)

What it does: A minimal HTTP server. It accepts a connection, reads the raw HTTP request, extracts the requested path (defaulting to /index.html when the path is just "/"), reads the matching file out of a local www/ folder, and replies with a proper HTTP response, either 200 OK with the file's contents, or 404 Not Found if the file does not exist.

How we tested it: We pointed a browser at http://127.0.0.1:8080 and confirmed our styled index.html page loaded correctly, including a link through to a second page, about.html, served from the same www/ folder using the same code path.

3.4 Framing Server & Client (Port 7100)

What it does: This program exists to demonstrate a core TCP concept, TCP only guarantees a stream of bytes, not a stream of messages. Data sent in two separate calls can arrive merged together, and data sent in one call can arrive split across several reads. To handle this correctly, the server keeps a persistent per connection string buffer and only treats something as a complete message once it has found a newline character inside that buffer.

How we tested it: Our client deliberately sent "Hello" and " World\n" as two separate, deliberately delayed sends, and then sent two more messages joined together in a single send. The server correctly printed "Hello World" as one combined message, and correctly split the joined send back into two separate messages, proving the buffering logic handles both directions of the problem.

3.5 Threaded Server (Port 7200)

What it does: To stop one slow or silent client from blocking everyone else, this server spawns a dedicated std::thread for every client it accepts, and detaches that thread so it runs independently. Each thread simply echoes back whatever that one client sends.

How we tested it: We opened two separate ncat connections to the server at the same time and sent different messages from each. Both were echoed back correctly and independently, and the server printed a "New client connected, starting a worker thread" line for each one, confirming that a second client is served immediately rather than having to wait for the first to disconnect.

4. Multi-User Chat System (Port 6000)

This was the main part of the assignment, it is a single server that many clients can connect to at once, with support for nicknames, a live user list, and broadcast messaging.

4.1 Server design (chat_server.cpp)

Rather than one thread per client, the server uses a single threaded event loop built around select(). On every iteration, it builds an fd_set containing the listening socket plus every currently connected client, and select() blocks until at least one of them has data ready. This avoids the overhead of threads and the need for any locking around shared state, since only one thread ever touches the client list.

- New connections are accepted and given a default name (user1, user2, …)
- Each client has its own input buffer (inbuf) so a message that arrives in pieces across multiple recv() calls is only processed once a full line (ending in \n) has been assembled.
- /nick <name> - changes a user's display name, after checking it is 1–16 characters of letters, digits, or underscores, and is not already taken
- /who - sends the requesting client a list of every currently connected name

- /quit - closes that client's connection cleanly
- Any other line is broadcast to every other connected client as “Name: message”, and joins/leaves are announced the same way

4.2 Client design (chat_client.cpp)

The client needs to do two things that would otherwise block each other: read what the user is typing, and continuously listen for incoming messages from the server. We solved this with two threads - the main thread loops on std::getline() and sends each line to the server, while a second, detached style thread (receiver) loops on recv() and prints anything the server sends, the instant it arrives.

4.3 Testing evidence

We tested the chat system with four simultaneous client windows rather than just one, specifically to prove that broadcasting actually reaches every other connected user and not just the sender. We verified, in order: that all four clients could join and appear in the server's log; that a message typed in one client's window appeared on every other client's screen; that /nick correctly updated a name everywhere, not just locally; that /who listed every connected user; that two clients typing messages back-to-back under time pressure still arrived whole and unmixed (the split message stress test); and that /quit disconnected a client and notified the rest.

5. Bonus - Crashing the Web Server

The bonus task asked us to find and demonstrate a real weakness in one of the provided servers. We targeted web_server.cpp, which handles exactly one client at a time: it accepts a connection, calls recv_some() to read the request, responds, and only then loops back to accept the next client.

5.1 The weakness

That recv_some() call has no timeout. If a client connects and never sends any data, the server is left waiting on that one read forever - and because it is single threaded, it cannot get back to accept() and serve any other visitor in the meantime. A single silent connection is enough to freeze the entire server for every other user.

5.2 Proof of concept

We wrote a small C++ program, crashbonus.cpp, that connects to the web server on port 8080 and then deliberately sends nothing for 30 seconds before closing the connection:

```
#include "net.hpp"
#include <iostream>
#include <thread>
#include <chrono>

int main() {
    net::init();
    std::cout << "Connecting to the web server...\n";
    sock_t s = net::tcp_connect("127.0.0.1", 8080);
    std::cout << "Connected - staying silent for 30 seconds. "
                 "Server should now be stuck.\n";

    std::this_thread::sleep_for(std::chrono::seconds(30));

    net::close_socket(s);
    net::cleanup();
    std::cout << "Done, socket closed.\n";
    return 0;
}
```

With web_server.exe running, we ran crashbonus.exe in a second terminal and, immediately after it printed “staying silent,” opened a browser and requested http://127.0.0.1:8080 from a third window. The page sat there loading and never returned, because the server was still blocked waiting on the silent connection and could not accept the browser's request. Once the 30 second sleep finished and

crashbonus.exe closed its socket, the server immediately unblocked and the browser's request was served normally.

5.3 Why this happens, and how it could be fixed

The root cause is that a single threaded server with a blocking, timeout free recv() can only ever make progress with one connection at a time, and a malicious or simply broken client can hold that one slot open indefinitely. A real fix would involve either setting a receive timeout on the socket so a silent client gets dropped automatically, or restructuring the web server the same way we restructured the chat server using select() or one thread per connection so that one stuck client can never block anyone else.

6. Compilation Commands

All programs were compiled with MinGW g++ on Windows. Every program links the Winsock library with -lws2_32, and any program using std::thread (the framing client, the threaded server, and both chat files) additionally needs -std=c++17.

```
g++ daytime_server.cpp -o daytime_server.exe -lws2_32
g++ daytime_client.cpp -o daytime_client.exe -lws2_32

g++ echo_server.cpp -o echo_server.exe -lws2_32
g++ echo_client.cpp -o echo_client.exe -lws2_32

g++ web_server.cpp -o web_server.exe -lws2_32

g++ frame_server.cpp -o frame_server.exe -lws2_32 -std=c++17
g++ frame_client.cpp -o frame_client.exe -lws2_32 -std=c++17

g++ thread_server.cpp -o thread_server.exe -lws2_32 -std=c++17

g++ chat_server.cpp -o chat_server.exe -lws2_32 -std=c++17
g++ chat_client.cpp -o chat_client.exe -lws2_32 -std=c++17

g++ crashbonus.cpp -o crashbonus.exe -lws2_32 -std=c++17
```

7. Verification & Testing Evidence

The screenshots below are a representative sample of our testing; the complete set, including every individual test described in Section 4.3, is saved in the screenshots/ folder of our GitHub repository.

7.1 Daytime Server & Client

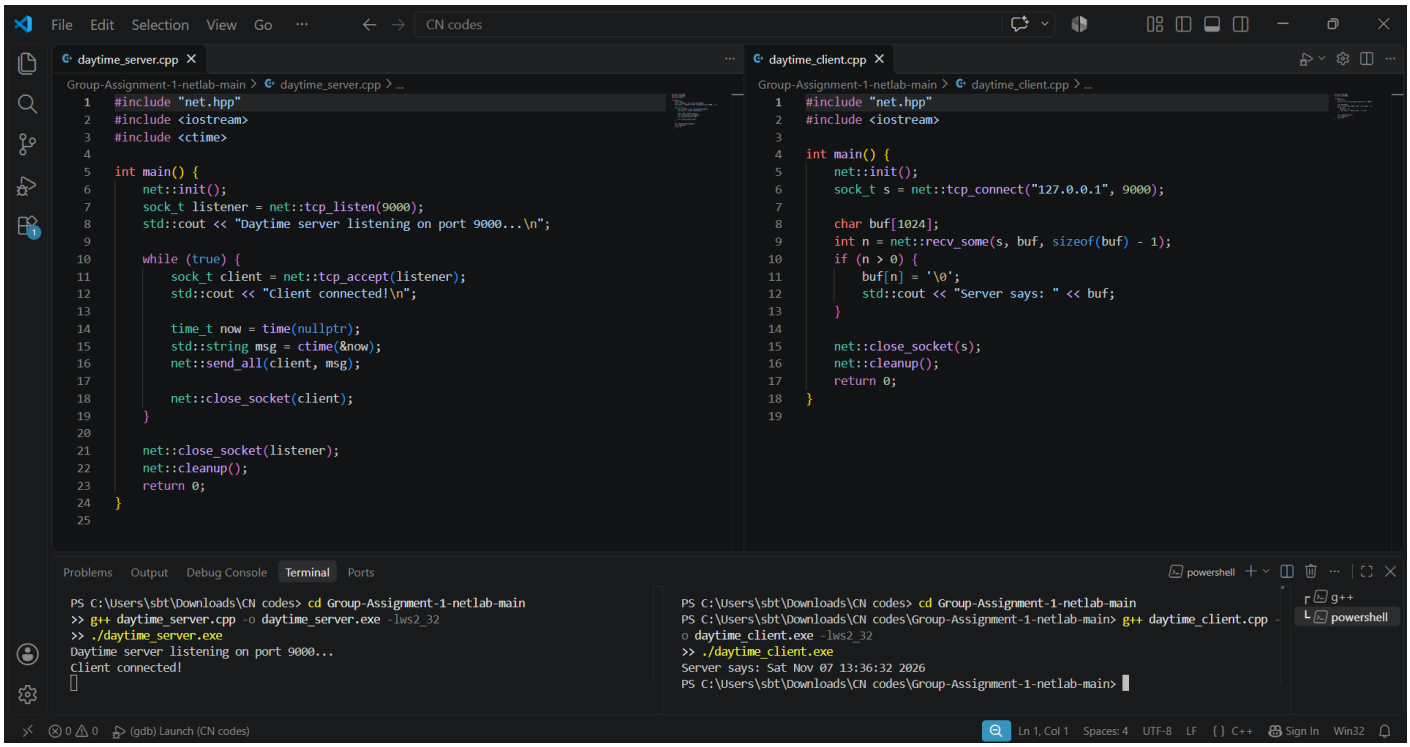


Figure 1 - Server listening and responding with the current date/time; client printing the received message.

7.2 Echo Server & Client

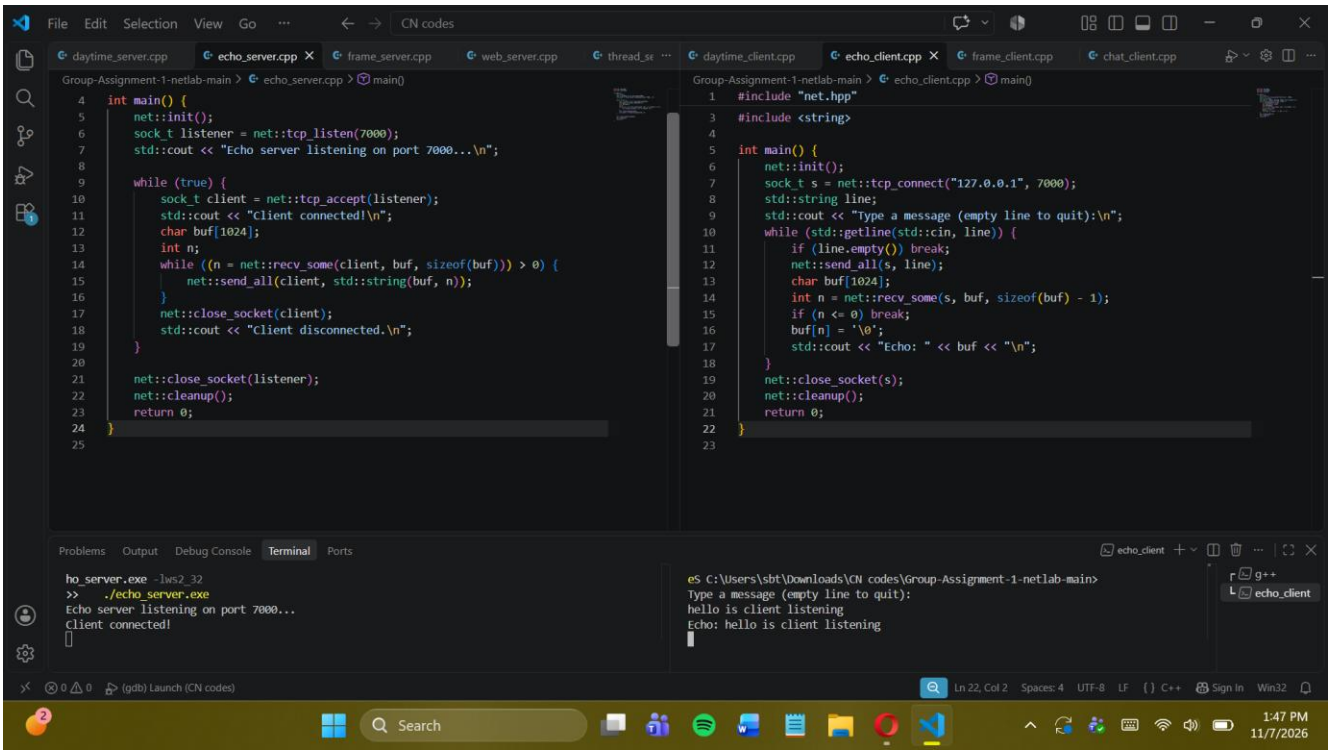


Figure 2 - A message typed in the client is echoed back unchanged by the server.

7.3 Static Web Server

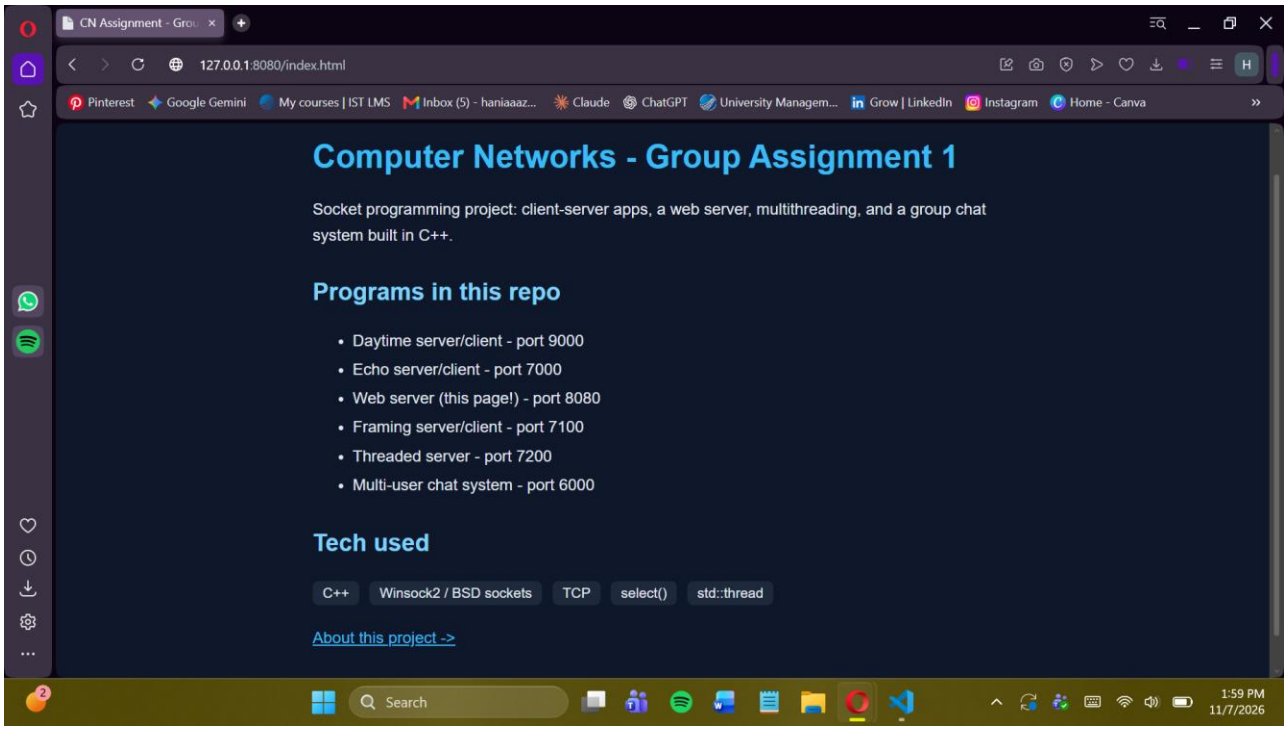


Figure 3 - Server started and listening on http://127.0.0.1:8080.

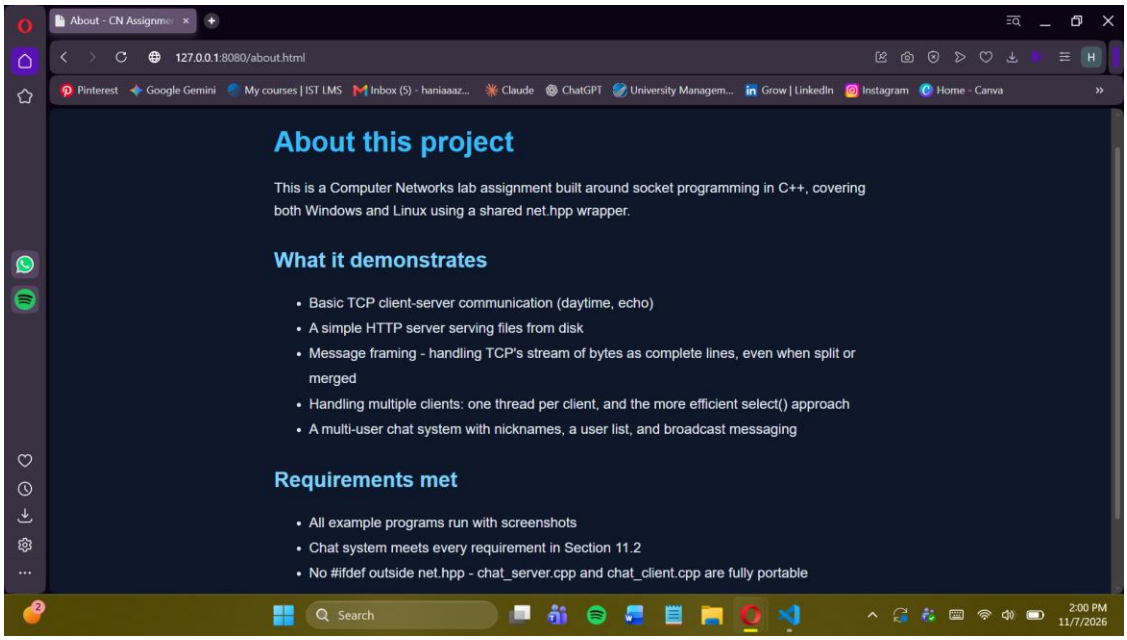


Figure 4 - index.html loaded correctly in the browser.

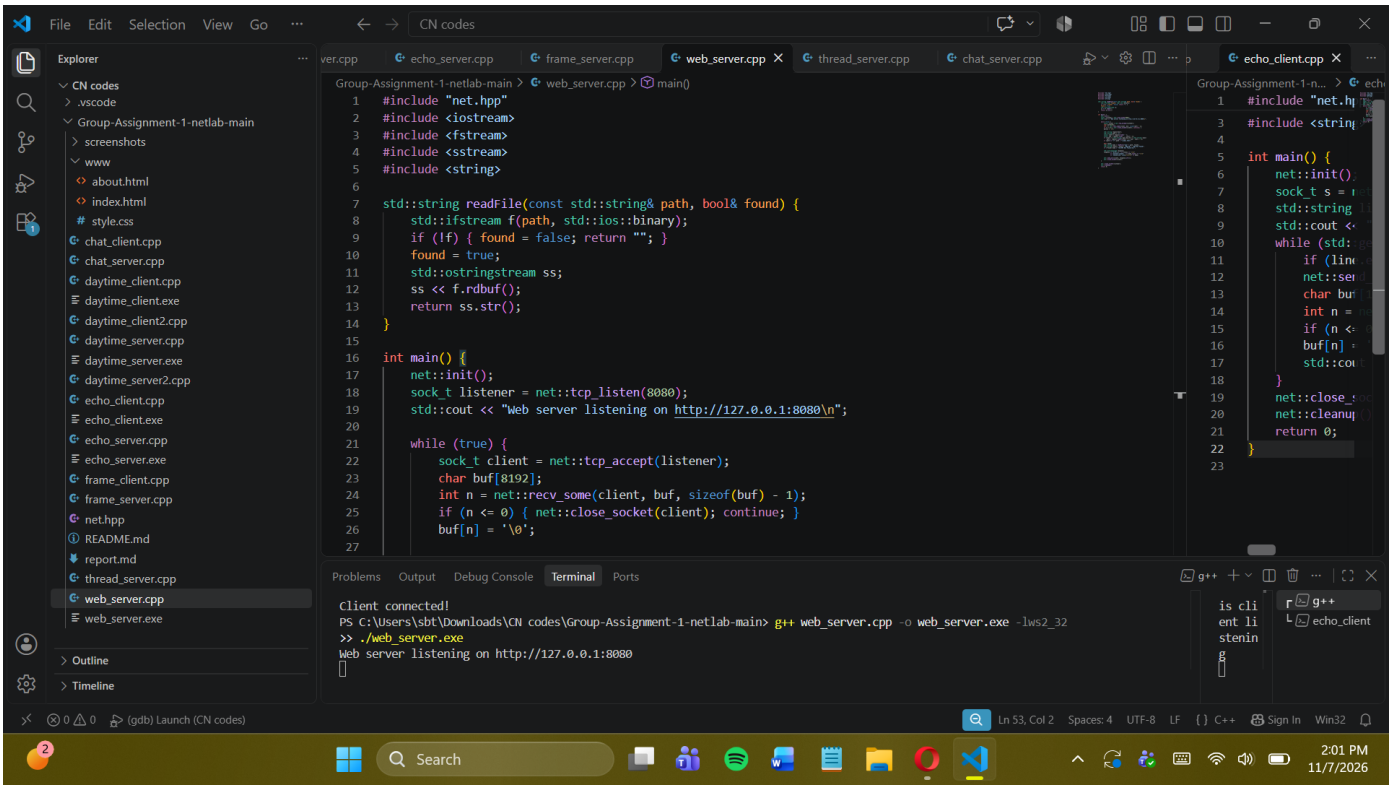


Figure 5 - about.html, served from the same www/ folder, loaded via the link on the home page.

7.4 Framing Server & Client

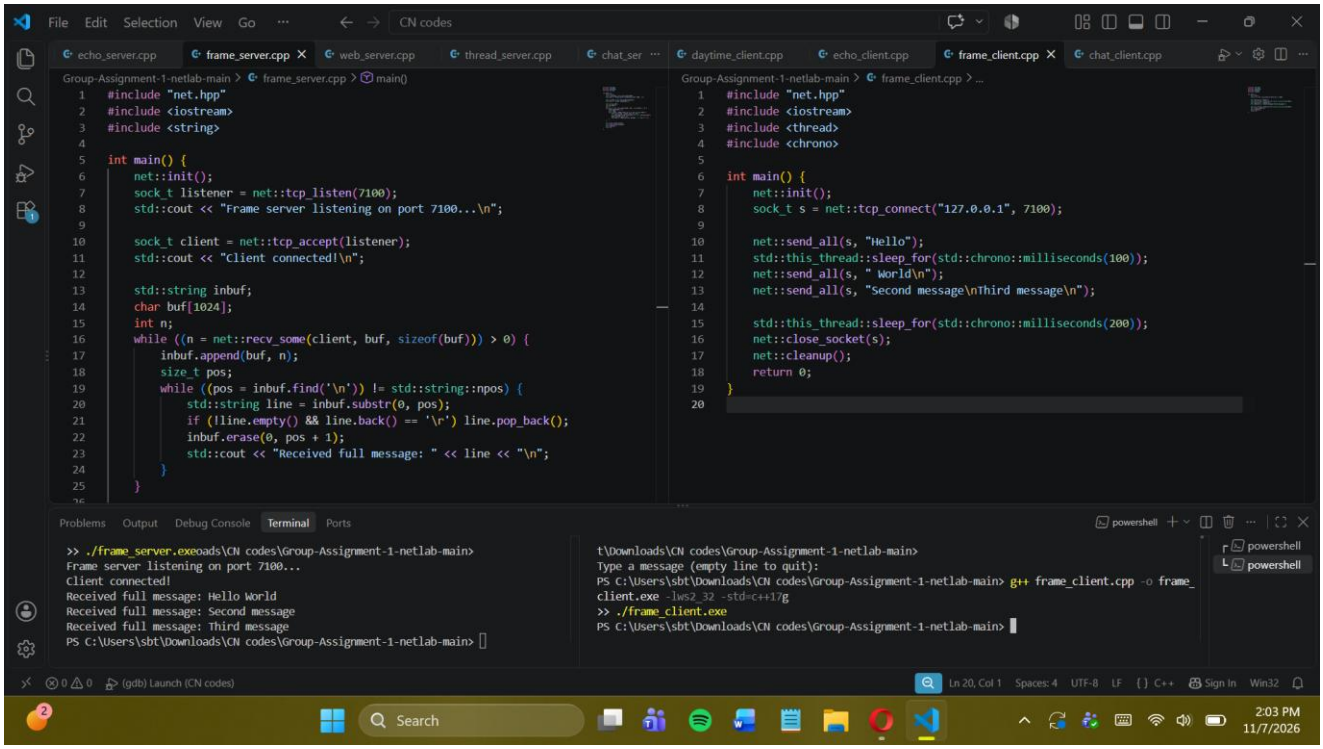


Figure 6 - Messages sent split across two sends ("Hello"/" World") and messages sent merged in one send both reassembled correctly into the right number of complete lines.

7.5 Threaded Server

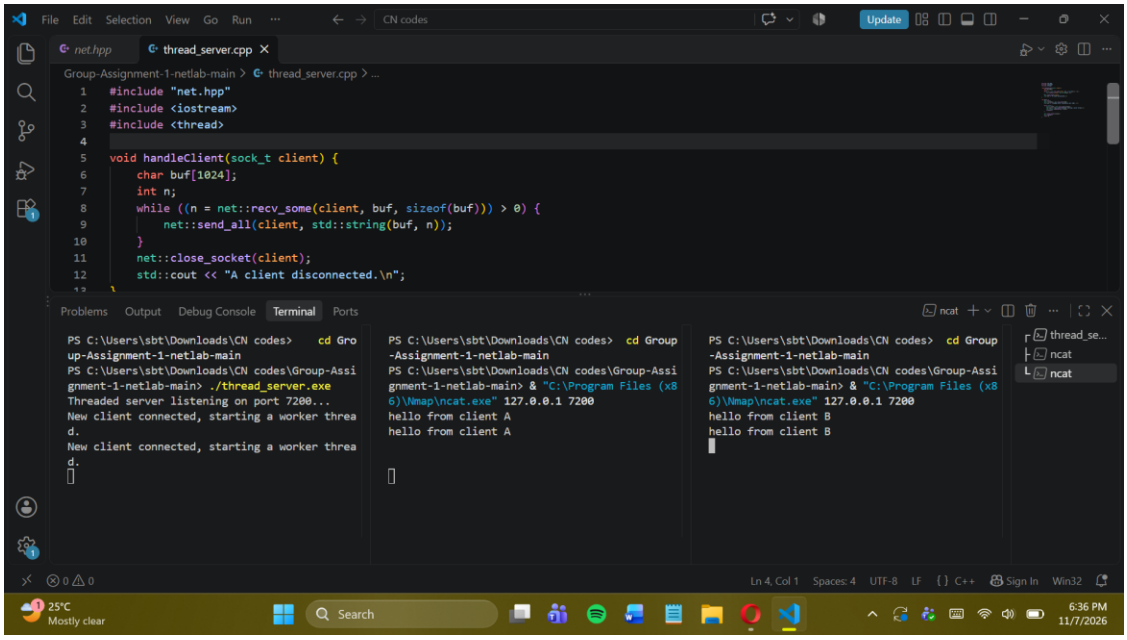


Figure 7 - Two simultaneous ncat clients connected at once; the server spawned a worker thread for each and echoed both independently.

7.6 Multi-User Chat System

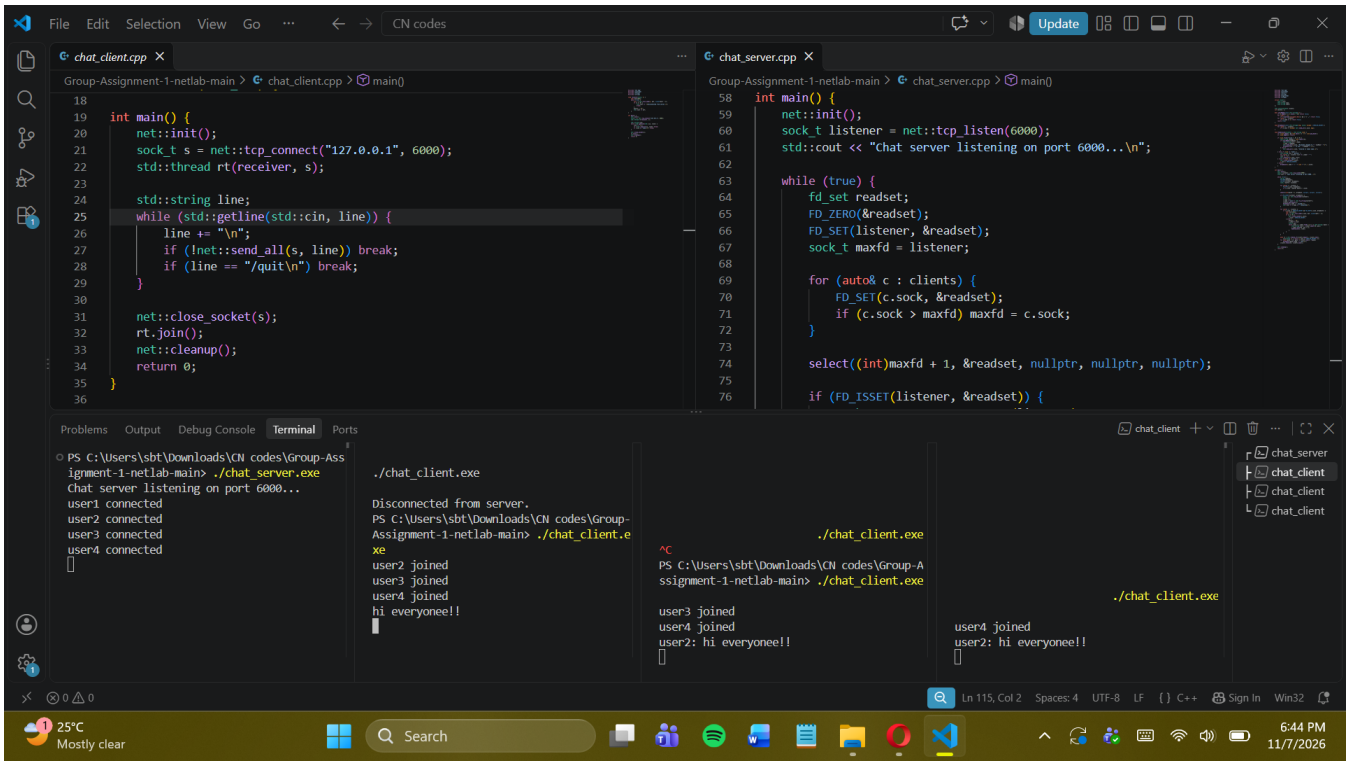


Figure 8 - Four clients connected; a broadcast message, a /nick change, and /who all visible across every client window.

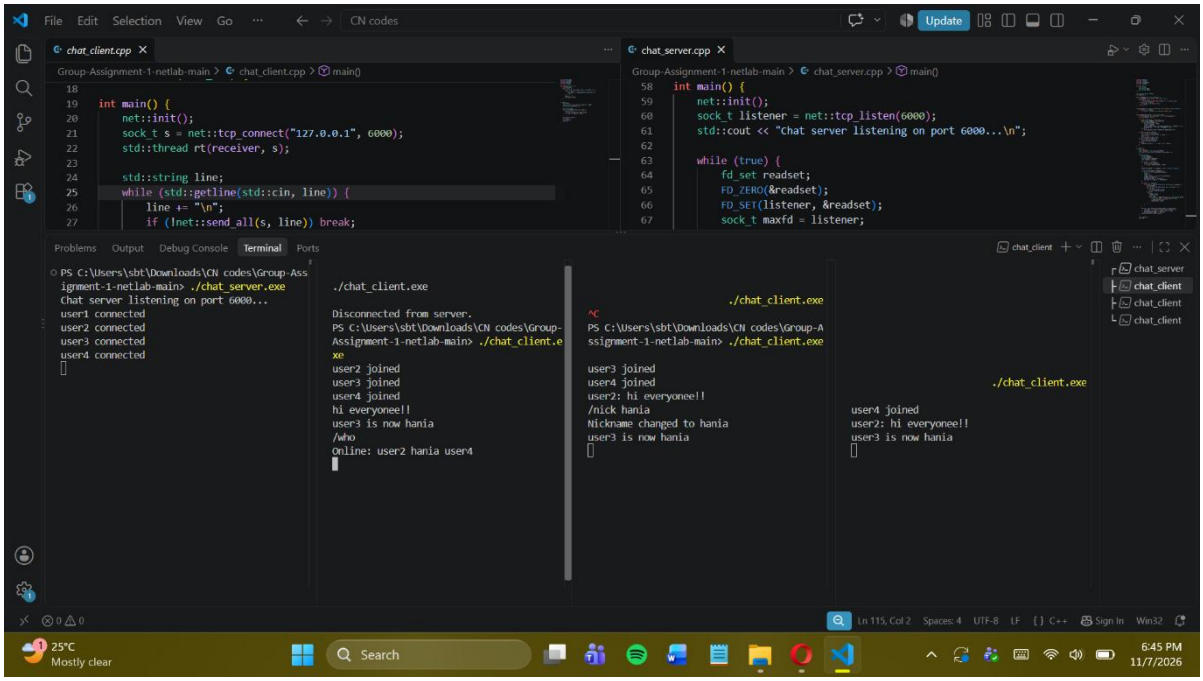


Figure 9 - Two more clients joining mid-session and receiving the same broadcast messages as the original clients.

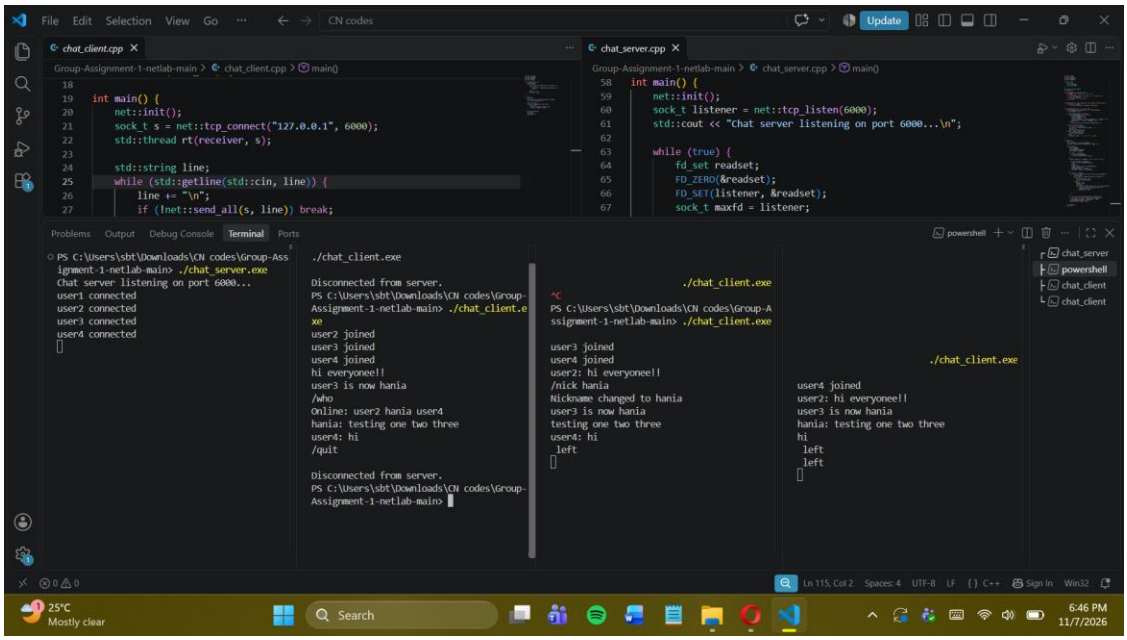


Figure 10 - select() loop and per-client FD_SET logic shown alongside the live 3-terminal test.

7.7 Bonus - Crashing the Web Server

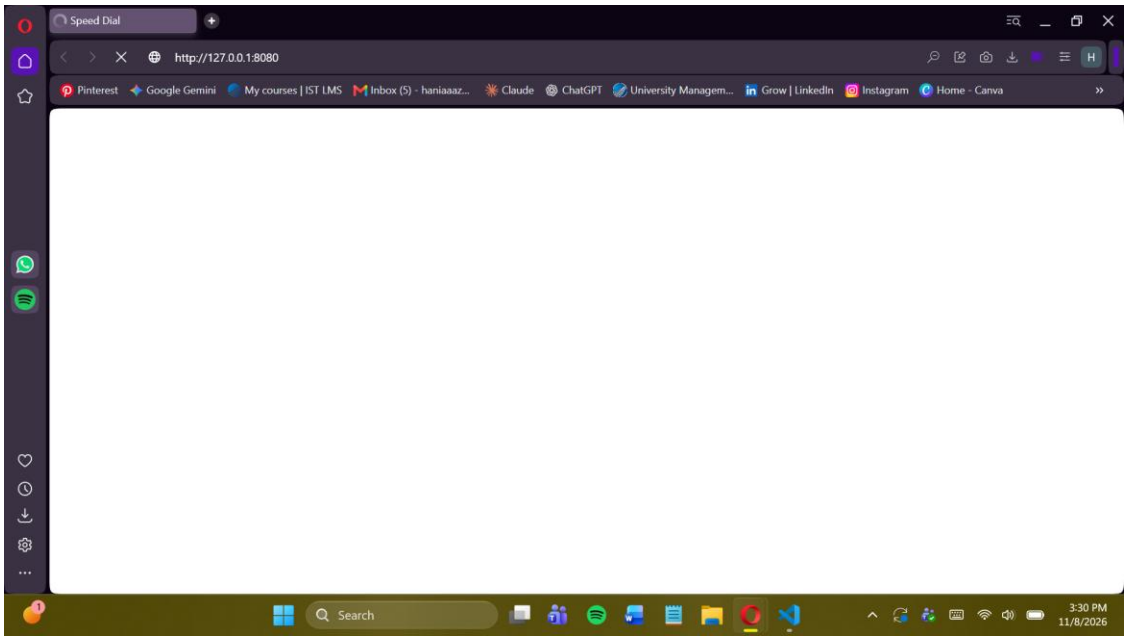


Figure 11 - crashbonus.exe holding a silent connection open; the server shows no further output while blocked.

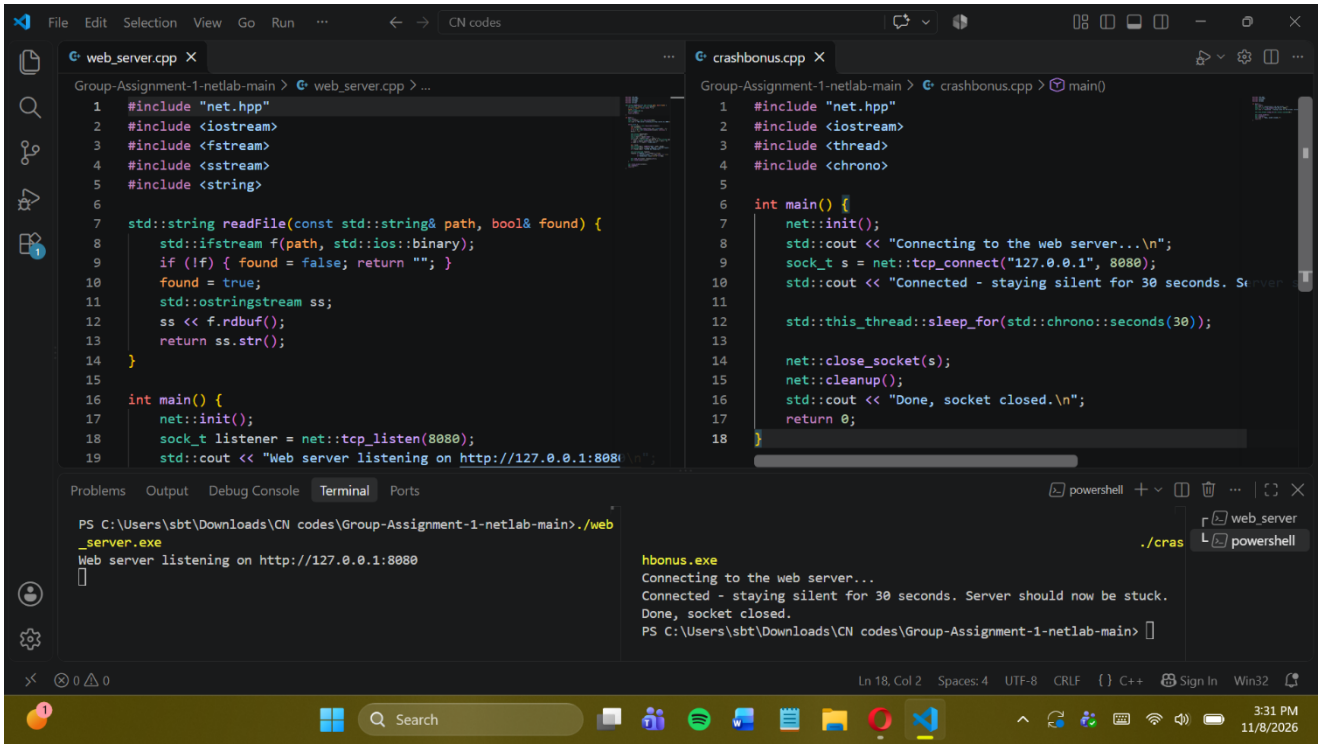


Figure 12 - Browser request to the same server left hanging while the server is blocked on the silent connection.

8. Challenges We Faced

A few real setup issues slowed us down and are worth recording here since they are easy mistakes to repeat. The repository's net.hpp and every .cpp file had initially been pushed to GitHub as empty placeholder files, so our first compiles failed with confusing “undefined reference to WinMain” errors until we realised the files simply had no code in them yet. We also repeatedly lost track of which terminal was sitting in which folder, since every new VS Code terminal resets to a default directory rather than following an already open one, which produced several “No such file or directory” errors that were really just a missing cd command.

None of these were conceptual misunderstandings of sockets themselves- they were the kind of practical friction that comes from a five person team sharing one repository for the first time, and resolving them was a useful exercise in its own right.

9. Requirements Checklist

The table below maps our implementation back to the assignment's stated requirements.

Req.	Description
Part 1	All Tutorial 1–8 example programs run successfully with screenshots (daytime, echo, web, framing, threaded
R1–R4	Chat server accepts multiple clients, assigns default names, and tracks each connection
R5	Per client input buffering correctly reassembles split/merged messages
R6	/nick validates the name (1–16 letters/digits/underscore) and rejects duplicates
R7	/who lists all currently connected users
R8	/quit disconnects the client cleanly and notifies the rest
R9	Broadcast messaging in the form “Name: message”, including join/leave notices
R10	Client uses two threads - one for sending, one for continuously receiving
R11	No #ifdef outside net.hpp - chat_server.cpp and chat_client.cpp are fully portable
Part 3	Public GitHub repository containing all source files, net.hpp, and screenshots
Bonus	A documented, working crash/weakness demonstration against the web server

10. Conclusion

This assignment took us from a two message daytime exchange to a server capable of juggling several live chat clients at once, and in doing so gave us a much more concrete understanding of what a “connection” actually is at the socket level boundaries, and

how handling more than one client at a time is a genuine design decision rather than something that happens automatically.

Building net.hpp first, before anything else, turned out to be the right call, it meant every program after it could be written once and were never a Windows/Linux conversation. The select() based chat server was the most rewarding part to get right, since watching four terminals broadcast to each other correctly, including under the stress test, was the clearest possible proof that the framing and buffering logic actually worked, not just in theory.

GitHub Repository:

https://github.com/Farwa705/Group-Assignment-1-netlab/tree/main

